

Porting `igsca_tree` / `igsca_ctree` into `doTrees()`

What is missing, what is wrong, and what to change — branch `igscaTrees`

1 Scope and method

This reviews the port of `dev/igsca/tree_template/igsca_tree.R` and `igsca_testers.R` into `R/postestimate_doTrees.R` + `R/helper_doTrees.R`, and the new `tests/testthat/test-postestimate_doTrees.R`.

Every finding below was **executed**, not read off. All five influence values were run against `tests/testthat/data/igscaTrees.Rdata` ($n = 1000$, covariates `z_true` (2-level factor), `noise_1`, `noise_2`) under `devtools::load_all()`, with reduced `R_test/maxdepth/max_cuts` where the default control was too slow to finish. Verbatim error text is quoted.

Revision history. The first pass found the partition family unrunnable. Since then:

- **Second pass** — `idx_permutation()` and `ndt_dists()` were added (`R/helper_doTrees.R:928, :959`) and `devtools::document()` was run. Verified in B1 and P1.
- **Third pass** — `splitter` became a `match.arg()` string and the test file was updated to match. Verified in B2 across all 17 configurations the tests exercise, plus both guard paths.
- **Fourth pass** — the influence function moved to its own `influence_fn` local (B2), and S1 was fixed: `argmax_split()` now counts failed kernel scans, `doTrees()` warns on a wholly dead kernel, and the three tests of section 5.4 are implemented and passing.
- **Fifth pass** — S2's dropped `cc$bonferroni` line was restored and verified on both distributions. Section 3.2.1 now answers the `testtype` TODO properly, including why the asymptotic route is genuine COIN inference and why partykit's "Bonferroni" is really Šidák.
- **Sixth pass** — S3 fixed: leaves are refit after growing and attached to the terminal nodes, so `coef()` and `plot()` work and the tree finally carries a per-leaf IGSCA fit. Inner-node objects are kept, with an opt-in switch.
- **Seventh pass** — T4 first, then T1–T3, which is the only workable order: until the controls are split by branch the suite cannot finish, and T2's node counts fail for reasons that have nothing to do with correctness. The file now runs in **2 min 05 s** at `FAIL 0 | WARN 0 | SKIP 0 | PASS 85`, against an estimated ~ 16 h before.
- **Eighth pass** — §3.2.1's open question settled: the `cc$bonferroni` override was measured to be a no-op and deleted, with both TODOs. T5 closed — every collector field and the scan cache now have tests, in `test-postestimate_doTrees.R` and `test-helper_doTrees.R` respectively. `FAIL 0 | WARN 0 | SKIP 0 | PASS 121` across the two files, still 2 min 05 s.
- **Ninth pass** — P5 fixed, and with it P4, P6 and P3's `doTrees` half: `doTrees()` now takes a **fitted** `cSEMResults` and replays that fit's own `csem()` arguments at every node, so no estimator is hard-coded anywhere. The snapshots did not move, which is the evidence the replay reproduces the old hard-coded call exactly. `PASS 150` in 2 min 25 s. The reasoning behind the API is recorded in section 9.
- **Tenth pass** — P2, then P1, then P3, in that order because each one shrinks the next: the `:::` prefixes went first, then 22 of the 26 exports became `@noRd` (deleting 23 `.Rd` files), leaving only four public symbols to document properly. `devtools::check_man()` is clean for every file the port touches. **All findings in this document are now closed** except section 5's 5.1–5.3 and 5.6.
- **Eleventh pass** — §5's 5.1–5.3 built, all three in the order the document proposed them: the kernels are now exercised one node at a time (finite statistic, admissible and discriminating cutpoint, and `partition_select()`'s `.Machine$double.eps` floor), and the undocumented `cc$splitfun` contract is asserted by counting the kernel's invocations. `FAIL 0 | WARN 0 | SKIP 0` at `PASS 130` in 2 min 13 s (`test-postestimate_doTrees.R`) and `PASS 37` in 10 s (`test-helper_doTrees.R`). The snapshots did not move.

Every finding in this document is now closed. Each `influence × splitter` combination runs, the three silent failures are fixed, all 17 configurations are asserted to have **actually split** rather than merely not thrown, every diagnostic the collector carries is asserted somewhere, and the package surface is four exported symbols with real documentation behind them.

What remains is not a finding but a **fixture**. 5.1–5.3 now localise a fault to a single kernel, to a single cutpoint, or to the `cc$splitfun` wiring — but they compare the kernels against each other, never against a known answer, because this fixture has no numeric covariate with a true threshold in it. Section 5.6 is that missing fixture, and it is worth building only if the mixed pairs are a real study arm.

1.1 Status at a glance

ID	Finding	Status
B1	<code>idx_permutation()</code> / <code>ndt_dists()</code> not ported	resolved
B2	<code>splitter</code> is a function in one branch, a string in the other	resolved
S1	Broken split kernel degrades to a stump with no diagnostic	resolved
S2	<code>control\$bonferroni</code> silently ignored on the <code>ctree</code> path	resolved
S3	<code>coef()</code> / <code>plot()</code> empty for any tree that splits	resolved
T1	Snapshots stale (<code>trees_out</code> vs <code>trees_mx</code>)	resolved
T2	<code>length(tree) == 5</code> is a disguised structural assertion	resolved
T3	<code>expect_no_error()</code> cannot see the real failure mode	resolved
T4	Default control makes the partition tests unrunnable	resolved
T5	Collector diagnostics and the scan cache are untested	resolved
P1	NAMESPACE stale, then over-exported	resolved
P2	<code>cSEM:::</code> / <code>cSEM::</code> into the package's own namespace	resolved
P3	Roxygen placeholders; wrong <code>@return</code>	resolved
P4	Argument naming departs from package convention	resolved
P5	<code>fit_csem()</code> hard-codes the estimator	resolved
P6	No input validation	resolved

The test file now carries two controls chosen per branch, five regenerated snapshots, one named `test_that()` per `influence × splitter` configuration — each asserting the tree grew rather than merely returned — a block per collector diagnostic, and the `cc$splitfun` contract test. `test-helper_doTrees.R` carries the node-level half: the kernel statistics, cutpoint choice, the log-scale floor, the scan cache and `partition_stat()`'s failure accounting. All that is left of section 5 is 5.6, the fixture that would make a **wrong** cutpoint checkable at all.

2 Resolved

2.1 B1. `idx_permutation()` and `ndt_dists()` are now available

Location: R/helper_doTrees.R:928, R/helper_doTrees.R:959

Both helpers now exist, and both satisfy the contracts their call sites require. This was the sole blocker on the partition family; it is cleared.

`idx_permutation()` — orientation is correct. The returned matrix is $R \times n$ (`nrow = R`, `ncol = n`), which is what `permutation_pvalue()` needs when it indexes `goes_left[perm[r, 1]]`. Had it come back transposed, the result would have been a recycled-length nonsense null distribution rather than an error, so this is worth having confirmed explicitly. The sample-without-replacement-within-stratum construction matches `boot::permutation.array` as the docstring claims.

`ndt_dists()` — names and argument order match the call site. It returns `c(DGc, DGi, DLc, DLi)`, and `partition_stat()` reads `unname(ds[stat_kind])` for `stat_kind` in "DGi" / "DLi". The (`Sc_pool`, `Si_pool`, `mga_fit`) signature lines up with `ndt_dists(coll$ndt_pools$Sc, coll$ndt_pools$Si, mga$fit)` at R/helper_doTrees.R:635.

Verified end to end. The partition family, previously unrunnable, now grows trees. At `R_test = 24L`, `max_cuts = 3L`, `maxdepth = 1L`, `minbucket = 200L` (14.8 s):

```
Fitted party:
[1] root
|   [2] z_true in 1: *
|   [3] z_true in 2: *

      z_true      noise_1      noise_2
statistic 0.04315312 0.001707126 0.001593087
p.value   0.04000000 0.280000000 0.083333333
criterion -0.04082199 -0.328504067 -0.087011377
```

The DLi/DGi kernels agree exactly with the reference implementations used to pre-verify the path in the first pass (`argmax` cutpoint on `noise_1 = -1.69654` for both, `-0.231765` for FIT), so the shipped `ndt_dists()` computes what that analysis assumed.

Two loose ends, both minor:

- **The `strata` argument is dead.** `idx_permutation(n, R, strata = NULL)` implements stratified permutation, but `permutation_pvalue()` (R/helper_doTrees.R:699) calls `idx_permutation(n, R)` with no `strata`. If stratified permutation within the node was intended — which would be the natural thing to want when the node is already conditioned on a covariate — it is not wired up. If it was not intended, drop the parameter rather than leave an untested branch in the package.
- **The provenance comment is now wrong.** R/helper_doTrees.R:535 still says these come from R/MGA/csem_test_helpers.R. They are defined 400 lines below it. (`dev/igsca/tree_template/csem_test_helpers.R` is now present but untracked.)

2.2 B2. `splitter` is now a single `match.arg()` string

Location: R/postestimate_doTrees.R:16--29

Implemented as recommended — one string-valued argument with an explicit "native" level, resolved once before either branch:

```
doTrees <- function(data, model, covariates,
                    influence = c("mat", "vec", "FIT", "DLi", "DGi"),
                    splitter   = c("native", "FIT", "DLi", "DGi"),
```

```

        control    = igsca_tree_control() {
influence <- match.arg(influence)
splitter  <- match.arg(splitter)

split_fn <- switch(splitter,
  native = NULL, FIT = split_max_fitdiff,
  DLi = split_max_dli, DGi = split_max_dgi)

```

split_fn is consumed by the ctree branch at :117 and by grow_extree() at :221. Both original failure modes are gone, and the structural hazard behind the second one is closed: match.arg() guarantees a length-1 character, so splitter %in% c(...) can no longer evaluate to logical(0) and slip through stopifnot() vacuously.

The retained stopifnot() at :167 is **not** redundant — it now does real work, rejecting "native" on the partition path, where there is no engine-native scan and a NULL kernel would die inside argmax_split()'s tryCatch and silently yield a stump. Both guards produce readable errors:

```

# (arguments as of P5's rename; the guards themselves are unchanged)
doTrees(res, covs, .influence = "FIT", .splitter = "native")
#> Error: splitter should be any one of 'FIT', 'DLi' or 'DGi' when the influence
#>      (selector) function is one of 'FIT', 'DLi' or 'DGi'

doTrees(res, covs, .influence = "mat", .splitter = "nope")
#> Error: 'arg' should be one of "native", "FIT", "DLi", "DGi"

```

Verified sweep. All 17 configurations the test file exercises now run, none error, and every one produces a split:

Configuration	width	n_fail (node/res)	time
mat × native	3	0 / 0	0.9 s
mat × {FIT, DLi, DGi}	3	0 / 2	1.7–2.0 s
vec × native	3	0 / 0	0.3 s
vec × {FIT, DLi, DGi}	3	0 / 0	1.5–1.8 s
{FIT, DLi, DGi} × {FIT, DLi, DGi}	2	0 / 0	14.0–15.3 s

(ctree rows at R_test = 50, maxdepth = 2, max_cuts = 8; partition rows at R_test = 24, max_cuts = 3, maxdepth = 1, minbucket = 200. The n_fail_resample = 2 on the mat mixed rows is ordinary non-convergence at extreme cutpoints, not a fault.)

Residual nits: both since fixed. The obsolete “add a switch statement” TODO has been deleted, and the mid-function rebinding of influence is gone — :45 now assigns the influence function to its own local, influence_fn, read at :79:

```

influence_fn <- switch(influence, mat = influence_mat, vec = influence_vec)
...
h <- influence_fn(E)

```

influence therefore stays a character for the whole call, so the else if (influence %in% c("FIT", "DLi", "DGi")) at :155 now tests what it appears to test. (An intermediate revision named this local selector, which collided with the genuine extree selectfun of the same name at :158/:219 — two different roles under one identifier in one function body.

`influence_fn` avoids that: `selector` is now unambiguously the variable-selection function `partykit` means by the term.)

Re-running the full sweep after the rename gave results **byte-identical** to the pre-rename run — every `width` and `n_fail` value unchanged, differing only in wall-clock jitter. As an independent check that the `switch()` still maps to the right function, `influence = "mat"` and `"vec"` produce different trees, and their node-2 cutpoints (`-1.09783` on `noise_1`, `1.46613` on `noise_2`) reproduce the recorded snapshot exactly. That is a real cross-check rather than a coincidence: on the native path the cutpoint comes from `ctree`'s deterministic `maxstat` scan, so it is invariant to `R_test` and the seed.

3 Silent failures — all three now resolved

3.1 S1. A broken split kernel degraded to a stump with no diagnostic

Location: R/helper_doTrees.R:293--345 (argmax_split, warn_dead_splitter)

Status: fixed. `argmax_split()` wraps each kernel call in `tryCatch(..., error = function(e) NA_real_)`, which is right for a genuinely non-identified candidate partition but also swallowed **programming** errors: every candidate became NA, `any(is.finite(stats))` was FALSE, the loop nexted through every covariate, `argmax_split()` returned NULL, and partykit read that as “no admissible split”. A dead kernel and an unsplittable node were indistinguishable in the returned object — which is why the original `ndt_dists()` defect had to be found by hand rather than by the suite.

The `tryCatch` is unchanged (it still has to absorb non-convergent candidates). What changed is that the two cases are now counted apart. `argmax_split()` tracks whether the kernel was **evaluated at all** on this invocation, and only a scan that ran and yielded nothing is recorded as a failure:

```
stats <- vapply(cands, function(cc) {
  tryCatch(splitter(model, mf, subset, cc$goes_left, ctrl),
    error = function(e) NA_real_)
}, numeric(1))
## The kernel was actually evaluated on this covariate, so the invocation
## counts as a scan regardless of what came back.
scanned <- TRUE
if (!any(is.finite(stats))) next
collector$n_split_scan <- collector$n_split_scan + 1L
return(cands[[which.max(stats)]]$split)
}
if (scanned) {
  collector$n_split_scan <- collector$n_split_scan + 1L
  collector$n_fail_split <- collector$n_fail_split + 1L
}
NULL
```

The distinction is load-bearing: a node with no admissible partition exits via `if (!length(cands)) next` and never sets `scanned`, so it is **not** counted as a kernel failure. Both counters were added to `new_collector()` and both are surfaced on the returned tree:

```
attr(tr, "igsca_info")$n_split_scan # scans where the kernel actually ran
attr(tr, "igsca_info")$n_fail_split # ... of which produced nothing usable
```

A totally dead kernel — `n_fail_split == n_split_scan` — now also warns, via a new internal `warn_dead_splitter()` called from both branches of `doTrees()` just before the attribute is attached:

```
Warning: The 'DLi' split kernel produced no usable statistic at any of the 2
node(s) where it was scanned, so no split point could be chosen from it and the
tree was grown as if no split were admissible. Check that the kernel runs
standalone via partition_stat().
```

The warning deliberately fires only on total failure. A **partial** failure (some nodes fine, others not) is ordinary non-convergence and would be noise as a warning — it is visible in the counters instead, where it can be asserted on. `splitter = "native"` short-circuits the check and leaves both counters at 0, since no kernel is involved.

Tests are in section 5.4 and now pass.

3.2 S2. control\$bonferroni was silently ignored on the ctree path

Location: R/postestimate_doTrees.R:94--108

Status: fixed, then superseded. The fix was first made by restoring the dropped `cc$bonferroni <- isTRUE(control$bonferroni)` override. It now ships as the length-2 `testtype` of section 3.2.1 instead, and the override has been deleted — see that section for the measurement that retired it.

The original diagnosis stands: the override was never a no-op **at the time**. `ctree_control()` derives `bonferroni` from `testtype`, but the port computes `testtype` from `coin_distribution` first, so with the default `coin_distribution = "approximate"` it always sent `testtype = "MonteCarlo"` and `cc$bonferroni` was always `FALSE` — while on the **partition** path `grow_extree()` copies the whole control list into `ectrl`, so the same argument did take effect there. Verified fixed on both distributions (`R_test = 200`, three covariates):

		z_true	noise_1	noise_2
asymptotic	FALSE	5.31e-57	0.872895	0.810172
asymptotic	TRUE	1.59e-56	0.997947	0.993160
MonteCarlo	FALSE	0.000	0.890	0.845
MonteCarlo	TRUE	0.000	0.998669	0.996276

3.2.1 Answering the TODO: testtype vs bonferroni

The earlier draft of this section said `testtype` picks “permutation vs asymptotic”. That is half the story and worth stating properly, because `testtype` conflates **two** dimensions:

testtype	p-value from	Multiplicity
"Bonferroni"	asymptotic limiting distribution	adjusted
"Univariate"	asymptotic limiting distribution	none
"MonteCarlo"	nresample permutations	none
"Teststatistic"	no p-value — raw statistic is the criterion	n/a

Are the p-values not always permutation-based? No — and this is the crux. In the conditional-inference (COIN) framework “permutation test” describes the **null distribution**, not the **computation**. Strasser and Weber derived the conditional mean μ and covariance Σ of the linear statistic T under that permutation null **in closed form**. Standardising with them gives $c = (T - \mu)^\top \Sigma^+ (T - \mu)$, which is asymptotically χ^2 with $\text{rank}(\Sigma)$ degrees of freedom (`teststat = "quadratic"`). You can then read the p-value off that χ^2 limit **without ever drawing a permutation**. That is the asymptotic route, and it is partykit’s default.

Two measurements confirm no resampling happens on that path — same seed, `influence = "mat"`, root criteria:

approximate, R_test = 50 :	z_true 0.00	noise_1 0.74	noise_2 0.82
approximate, R_test = 200:	z_true 0.000	noise_1 0.890	noise_2 0.845
asymptotic, R_test = 50 :	z_true 5.31e-57	noise_1 0.872895	noise_2 0.810172
asymptotic, R_test = 200:	z_true 5.31e-57	noise_1 0.872895	noise_2 0.810172

First, `5.31e-57` is unreachable for **any** Monte-Carlo test at `R = 50` or `200`. Second, the asymptotic values are bit-identical across the two `R_test` settings — `nresample` is simply unused. So `coin_distribution = "asymptotic"` is a real setting with the same conditional null and the same μ , Σ ; it only swaps an empirical tail for a χ^2 tail. It is far cheaper, and it yields continuous p-values instead of a grid; Monte Carlo is the safer choice in small nodes where the asymptotics have not bitten.

Why the fix is still needed. `.extree_node()` applies the adjustment to whatever p-values it was handed, independently of how they were produced:

```
if (ctrl$bonferroni)
  sf$criteria["p.value", ] <- sf$criteria["p.value", ] *
    sum(!is.na(sf$criteria["p.value", ]))
```

So the two knobs **are** orthogonal at the point of use, even though `testtype` bundles them at the point of configuration — which is exactly why an explicit assignment was required **while** `testtype` was being computed as a single string.

Two details worth knowing. That row holds $\log(1 - p)$, not p , so multiplying by k gives $\log((1 - p)^k)$ — the adjustment is Šidák, $1 - (1 - p)^k$, not Bonferroni's kp . It agrees with Bonferroni to first order (hence the exact $\times 3$ on `z_true` above) but not for large p : $1 - (1 - 0.872895)^3 = 0.997947$, matching the table exactly. Second, `partykit` **does** support asking for both dimensions at once — `ctree_control()` accepts a length-2 `testtype`:

```
testtype <- match.arg(testtype, several.ok = TRUE)
if (length(testtype) > 1) {
  stopifnot(all(testtype %in% c("Bonferroni", "MonteCarlo")))
  ttesttype <- "MonteCarlo"
}
... bonferroni = "Bonferroni" %in% testtype
```

`ctree_control(testtype = c("Bonferroni", "MonteCarlo"))` is identical to the manual override on every non-function component (verified; the function-valued slots differ only by closure environment). So the mapping can drop the override entirely and stay inside the documented API:

```
testtype <- c(
  if (isTRUE(control$bonferroni)) "Bonferroni",
  if (control$coin_distribution == "approximate") "MonteCarlo"
)
if (is.null(testtype)) testtype <- "Univariate"
```

which reaches all four combinations — MC± adjustment, asymptotic ± adjustment — with no post-hoc surgery on the control object.

3.2.2 The override was left in on top of it, and was a no-op

Resolved. An intermediate revision adopted the mapping above **and kept** the `cc$bonferroni <- isTRUE(control$bonferroni)` line beneath it, with the TODO comments that motivated both still attached. The two lines were not both needed, and the leftover made it look as though `ctree_control()` could not be trusted to derive the flag.

Measured across all four combinations, the override changed nothing:

bonferroni	dist	testtype passed	-> cc\$bonferroni
FALSE	approximate	MonteCarlo	FALSE
FALSE	asymptotic	Univariate	FALSE
TRUE	approximate	Bonferroni + MonteCarlo	TRUE
TRUE	asymptotic	Bonferroni	TRUE

`identical(cc_with_override, cc_without_override)` is TRUE in all four cases, so the assignment could only ever have re-written the value "Bonferroni" %in% testtype had already produced. It has been deleted, along with both TODO comments, and replaced with a comment stating the mapping.

Two further things were checked before removing it. `criterion` stays the length-1 "p.value" under a length-2 `testtype` — `ctree_control()` derives it from the **collapsed** `testtype`, not the vector, so the length-2 form introduces no recycling anywhere downstream. And the four root-criteria matrices are **bit-identical** before and after removal while remaining distinct from each other (`R_test = 200`, `maxdepth = 1`), which is the evidence that `bonferroni` is still live rather than quietly lost:

root p.value		z_true	noise_1	noise_2
asymptotic	FALSE	5.31e-57	0.872895	0.810172
asymptotic	TRUE	1.59e-56	0.997947	0.993160
MonteCarlo	FALSE	0.000000	0.890000	0.795000
MonteCarlo	TRUE	0.000000	0.998669	0.991385

These are the same four signatures S2 recorded when the override **was** the fix, which is the point: the argument still reaches partykit by a different route. The Šidák identity holds exactly on both distributions — $1 - (1 - 0.872895)^3 = 0.997947$ and $1 - (1 - 0.890)^3 = 0.998669$.

3.3 S3. `coef()` and `plot()` were empty for any tree that splits

Location: `R/helper_doTrees.R` (`attach_leaf_fits`, `drop_inner_node_objects`, `coef.igsca_tree`, `plot.igsca_tree`); wired in at `R/postestimate_doTrees.R`:150--160 and :230--240

Status: fixed. Both methods read `nobs / objfun` out of `info_node()` at **terminal** nodes, but partykit only calls the trafo at nodes it **attempts to split**, so leaves arrived with `info = NULL`. `coef()` returned `NULL` for every tree with at least one split (it only appeared to work on stumps, where the root **is** the leaf) and `plot()` drew unlabelled boxes, because `sprintf("n = %s", NULL)` yields `character(0)` rather than erroring.

The deeper problem was not the methods: **the tree carried no per-leaf IGSCA fit at all**. A new `attach_leaf_fits()` refits the model once per leaf after growing and writes the result into the terminal nodes' own `info`, so both methods now work through the ordinary `info_node()` route rather than a side table.

Two properties were verified before choosing that design:

- `as.list()` / `as.partynode()` round-trips a grown tree **identically**, so rewriting only the leaf entries leaves the rest of the structure untouched.
- Terminal-node `info` is invisible to `print()` — partykit's `formatinfo_node()` falls back to `"*"` for list-valued `info` — and that holds even when the `info` carries a full `cSEMResults`. Attaching leaf fits therefore does not change printed output or disturb the snapshots.

Measured, `maxdepth = 2` on the `ctree` path and a `FIT × FIT` partition tree:

ctree	coef()	nobs	objfun	partition	coef()	nobs	objfun
	node 3	68	NA		node 2	500	3553.173
	node 4	432	3076.803		node 3	500	3438.003
	node 5	500	3438.003				

Both previously returned `NULL`. `coef()` also gained row names (node ids) and now emits an `NA` row instead of silently dropping a node — `rbind()` discards `NULL`s, which is precisely how the method used to collapse a whole tree to `NULL`. `plot()`'s default panel prints `?` rather than an empty box when a value is missing. `objfun` is the sum of squared casewise GSCA residuals, matching what the `ctree` trafo stores on inner nodes; on the partition path it is the **only** source of an SSR, since that trafo sets `objfun = NA_real_` throughout.

What the fix exposes. Leaf refits can fail, and now say so: a new `n_fail_leaf` counter joins the `igsca_info` attribute, and the failing leaf gets `converged = FALSE`, `objfun = NA`, `object = NULL`. The

`maxdepth = 2` tree above hit exactly that at node 3 ($n = 68$). This is worth knowing when choosing `minbucket`: with 13 indicators the default 30L admits leaves the model may not fit. The failure is **sample-dependent rather than a size threshold** — random subsets of $n = 30$ and $n = 68$ fitted while $n = 50$ did not — so it is a rising probability of inadmissible estimates in small leaves, not a hard floor. Raising `minbucket` for larger models is the cheap defence; `n_fail_leaf > 0` is the signal that it is needed.

Inner-node objects: kept, with an opt-in switch. Every inner node's info still holds a full `cSEMResults`, because `saveinfo = TRUE` persists whatever the trafo returned.

`drop_inner_node_objects()` removes those payloads after growing, leaving criteria, `nobs`, `objfun` and all leaf fits intact. It is **not** called by default; a commented-out line in each branch of `doTrees()` marks the switch:

```
## MEMORY OPTIMIZATION (opt-in): each inner node's info holds a full
## cSEMResults object, so a maxdepth = 3 tree keeps up to 7 of them.
## Uncomment the next line to drop them; leaf fits are unaffected.
##   ret <- drop_inner_node_objects(ret)
## Do NOT instead delete `object = ft$fit` from the ytrafo above: a
## mixed-pair splitter reads model$object while the tree is still growing,
## so removing it there silently disables every non-native splitter.
```

That last warning is the point of putting the switch here rather than at the trafo. Deleting `object = ft$fit` looks like the obvious optimisation and is the one place it must not be done: on the `ctree` path `partition_stat()` reads `model$object` for every candidate partition of a mixed-pair splitter, and on the partition path the selectors read it too — removing it there would take out every non-native configuration, silently, exactly the way S1 used to fail. Measured saving from the safe switch: $1.9 \rightarrow 1.1$ Mb (`ctree`, 2 inner nodes) and $1.2 \rightarrow 0.7$ Mb (`partition`, 1 inner node), with `coef()` unchanged afterwards.

4 Test-suite findings

The test files are the right shape — one per R file, matching the conventions in `MEMORY.md`, with the tree-level assertions in `test-postestimate_doTrees.R` and the node-level ones in `test-helper_doTrees.R`. This section records what changed and what was measured along the way. All five findings are closed, and so is section 5’s 5.1–5.3; the only remaining test work is 5.6, which needs a fixture that does not exist yet.

4.1 T1. Snapshots were stale — deleted and regenerated

Status: fixed. `_snaps/postestimate_doTrees.md` recorded the printed object under the name `trees_out` while the tests bind `trees_mx / trees_vec / trees_NPT_FIT / ...`; `expect_snapshot()` captures the **expression** as well as the output, so both existing blocks failed on the `Code` line alone and the three partition blocks had no snapshot at all. The stale file was removed (commit 5e268c95) and regenerated under the new controls — five snapshots, one per influence value.

Two properties were checked rather than assumed:

- **Reproducible across processes.** A second `devtools::test()` run in a fresh session gives `WARN 0` — no “Adding new snapshot”, no diff — so the `set.seed()` at the head of each block pins both the libcoin Monte-Carlo draw and `idx_permutation()`.
- **The reference tree is unchanged by T4.** The `mat` snapshot still splits at `noise_1 <= -1.09783`, the cutpoint recorded in B2 long before any of this, because the two **native** snapshot trees were deliberately left at the package defaults. The three partition snapshots are new and are all the same 2-node split on `z_true`.

One packaging detail: removing the stale file took the last tracked file out of `tests/testthat/_snaps/`, so the regenerated file is **untracked** and needs a `git add tests/testthat/_snaps/` before it protects anything.

4.2 T2. `length(tree) == 5` — replaced with assertions that say what they mean

Status: fixed. `length()` on a `party` dispatches to `length.party()`, which returns the **number of nodes**, not the length of a list, so the five `expect_true(length(x) == 5)` lines asserted “this tree has exactly 5 nodes” — a strictly weaker duplicate of the snapshot immediately below, breaking on any legitimate change to the control defaults or the RNG stream, for a reason the `# Dirty substitute for snapshot` comment actively misdirected you about. They also had to go before T4 could land: the reduced controls produce 2- and 3-node trees, so all five would have failed for reasons unrelated to correctness.

They are replaced by one helper, applied to all 17 configurations:

```
expect_grew <- function(tree) {
  expect_s3_class(tree, "igsca_tree")
  info <- attr(tree, "igsca_info")
  expect_identical(info$n_fail_full, 0L)
  expect_identical(info$n_fail_split, 0L)
  # width() returns a double, so expect_identical(..., 1L) would fail on type.
  expect_gt(partykit::width(partykit::node_party(tree)), 1)
}
```

Nothing now asserts a node count, which is what makes the per-branch controls safe to tune: `expect_grew()` holds at any `maxdepth`, and the exact shape of each tree is the snapshot’s job.

4.3 T3. `expect_no_error()` — split into one named test per configuration

Status: fixed. Each of the five `expect_no_error({...})` blocks wrapped two or three `doTrees()` calls, so a failure told you that **one** of them errored but not which — and, far worse, per S1 the

realistic failure is not an error at all: a broken kernel returns a plausible stump and the block passes. That is exactly what happened in the first pass, when the DLi and DGi mixed splitters were entirely non-functional and the suite reported green.

Each block is now a loop that **generates** one `test_that()` per splitter, so the test name carries the configuration even when the call errors outright:

```
for (sp in c("FIT", "DLi", "DGi")) {
  test_that(paste0("Matrix of Residuals selection splits on ", sp), {
    set.seed(12353)
    expect_grew(grow_tree("mat", sp, ctl_mixed))
  })
}
```

`grow_tree(influence, splitter, control)` is a one-line local wrapper around `doTrees()`; it collapses 17 seven-line call sites to 17 one-line ones without touching what `expect_snapshot()` captures.

These assertions have teeth, verified. Section 5.4’s mocked dead kernel makes exactly the `expect_grew()` assertions fail — `width == 1` and `n_fail_split > 0` — which is the regression the old blocks could not see.

Two counters were deliberately left out of `expect_grew()`, both for measured reasons rather than caution:

- `n_split_scan > 0` does **not** hold on the partition path. A matched selector/splitter pair reuses the selector’s scan through the cache (section 5.5), so `FIT × FIT` legitimately reports `0 / 0`; only the mismatched pairs rescan.
- `n_fail_resample == 0L` does not hold either: the `mat` mixed rows reach 2, the ordinary non-convergence at extreme cutpoints noted in B2.

4.4 T4. Two controls, one per branch — the suite finishes in two minutes

Status: fixed. The two paths differ in cost by an order of magnitude, because only the partition selectors run permutation p-values; a mixed splitter re-scans candidates but inherits COIN variable selection from libcoin. Measured on this fixture:

Configuration	Wall clock
one 2-group MGA fit	0.585 s
ctree native, defaults (<code>maxdepth = 3</code> , <code>R_test = 500</code>)	1.4 s
mixed splitter, <code>maxdepth = 2</code> , <code>max_cuts = 8</code> , <code>R_test = 50</code>	1.5–2.6 s
partition selector, <code>maxdepth = 1</code> , <code>max_cuts = 3</code> , <code>R_test = 24</code>	15.9 s
partition selector, <code>maxdepth = 2</code> , <code>max_cuts = 4</code> , <code>R_test = 24</code>	31.6 s
partition selector, one node at <code>max_cuts = 20</code> , <code>R_test = 500</code>	~ 15 min
partition selector, one tree at <code>maxdepth = 3</code> (default)	~ 1.8 h

All 17 calls used a bare `igsca_tree_control()`. The eight ctree-branch calls were fine at the defaults; the nine partition-branch calls were not — at ~ 1.8 h each the file was on the order of **16 hours**. What shipped:

```
ctl_mixed <- igsca_tree_control(R_test = 50L, maxdepth = 2L, max_cuts = 8L)
ctl_part  <- igsca_tree_control(R_test = 24L, max_cuts = 3L, maxdepth = 1L,
                                minbucket = 200L)
```

`ctl_part` is **shallower and cheaper than this section originally recommended**, on measurement. The proposed `maxdepth = 2L`, `max_cuts = 4L`, `minbucket = 100L` runs in 31.6 s and returns the **same 3-node tree** as the `maxdepth = 1L` control at 15.9 s: the children never split on this fixture, so the second level buys runtime and nothing else. Depth-2 recursion is exercised on the `ctree` branch by `ctl_mixed` instead, where it is nearly free.

`R_test` matters only for `ctl_part`, and is worth the comment it carries in the file.

`permutation_pvalue()` returns $(1 + k) / (R + 1)$, so the smallest attainable p-value is $1/(R + 1)$, and the split criterion is a **strict** inequality (`crit > logmincriterion` $\Rightarrow$ `p < alpha`). At `R_test = 19L` the floor is exactly $1/20 = 0.05$, which does **not** clear `alpha = 0.05`: the partition family needs `R_test >= 20L` or it silently returns stumps for reasons that have nothing to do with the data. `24L` clears it at $1/25 = 0.04$.

This does **not** apply to `ctl_mixed`. `libcoin`'s Monte-Carlo p-value is a plain proportion with no add-one — measured, $0.74 = 37/50$ and $0.890 = 178/200$ exactly — so `p = 0` is attainable at any `R_test` and there is no such floor on the `ctree` branch.

The two native snapshot trees keep the package defaults. They cost 1.4 s each, so there is nothing to buy by reducing them, and leaving them alone is what lets the `mat` snapshot stay comparable to every earlier measurement in this document (T1). `ctl_mixed` covers the six mixed `ctree` calls plus section 5.4's three blocks; `ctl_part` covers all nine partition calls.

Result: FAIL 0 | WARN 0 | SKIP 0 | PASS 85 in **2 min 05 s**, so no part of this file needs `skip_on_ci()`.

4.5 T5. The diagnostics the port exists to collect are now asserted

Status: fixed. Every field on `attr(tr, "igsca_info")` is now covered, plus the scan cache. The counters are the port's only window into **partial** failure — each is reached by a path that still returns a well-formed tree, so nothing else in the suite would notice one going wrong.

`n_split_scan` and `n_fail_split` were already pinned by section 5.4 and T2's `expect_grew()`. The four blocks added to `test-postestimate_doTrees.R` cover the rest:

Field	How it is reached, and what is asserted
<code>n_fail_full</code>	<code>try_fit</code> mocked to fail unconditionally. Asserts <code>1L</code> , <code>n_fail_node == 0L</code> , <code>root_criteria</code> <code>NULL</code> , and a stump — the collector's <code>root_seen</code> flag is the only thing separating this from the next row
<code>n_fail_node</code>	<code>try_fit</code> mocked to fail only below the full sample size, so the root fits and every child fails. Asserts <code>n_fail_full == 0L</code> , <code>n_fail_node > 0L</code> , and that the root test still ran (the tree split)
<code>root_criteria</code>	Structure (<code>colnames == covs</code> , all three rows present) plus the statistical claim the fixture supports: <code>z_true</code> wins the root criterion outright and neither <code>noise_*</code> clears <code>alpha</code>
<code>n_fail_leaf</code>	The default-control <code>mat</code> tree, where the <code>n = 68</code> leaf of S3 genuinely fails. Asserts <code>1L</code> , and that <code>coef()</code> reports it as an <code>NA objfun</code> row with its <code>nobs</code> rather than dropping the node

The `n_fail_leaf` block doubles as the only test of `S3`'s `coef()` fix: the regression it guards against is `rbind()` silently discarding a `NULL`, which is exactly how that method used to collapse a whole tree to `NULL`.

`n_fail_resample` and the scan cache are node-level and live in `test-helper_doTrees.R` — see section 5.5, now implemented. `partition_stat()`'s contract is that an unfittable candidate partition comes back as `NA_real_` with the counter bumped, **never** as an exception; anything that escapes there would make `SimDesign` redraw the whole replication.

5 Testing the mixed-splitter configurations

The mixed pairs (COIN variable selection + a model-comparison split point) are the configurations the suite is least able to check, because their failure mode is a **plausible-looking tree**, not an exception. T3 closed the crudest version of that gap — a wholly dead kernel now fails `expect_grew()` — and 5.1–5.3 below close everything else that is checkable without a new fixture. Three measured facts shaped what those tests could look like, and (b) is why the whole layer lives in `test-helper_doTrees.R` rather than at the tree level.

(a) **The splitfun wiring works, but it is an undocumented contract.** `doTrees()` plants its kernel in `cc$splitfun` and relies on `partykit` reading it back out of the control object. A counting wrapper confirms it fires — 9 kernel calls on a `maxdepth = 2`, `max_cuts = 8` tree (1 at the root for `z_true`, 8 at node 2 for `noise_1`) on `partykit` 1.2.29. The template’s note says “verified against `partykit` 1.2.28”, so this contract had already survived one upgrade unverified. 5.3 is the test that now pins it.

(b) **The fixture cannot falsify cutpoint choice at the root.** `z_true` is a 2-level factor, so `candidate_partitions()` returns exactly **one** candidate:

```
length(candidate_partitions(1, dat$z_true, dat$z_true, max_cuts = 20L, minbucket =
30L))
#> 1
length(candidate_partitions(2, dat$noise_1, dat$noise_1, max_cuts = 20L, minbucket =
30L))
#> 20
```

Every kernel is forced to return the same split at the root. Any root-level assertion about **which** cutpoint a mixed splitter chose is vacuous, which is why no amount of rephrasing the existing `expect_no_error()` would have worked.

(c) **Kernels do disagree, on numeric covariates.** Pinning `whichvar` to `noise_1` and running `argmax_split()` directly, under the `node_fixture()` 5.1 ships (`max_cuts = 8`, `minbucket = 100`, `noise_1` $\in [-2.744, 3.029]$):

```
FIT  cutpoint on noise_1 = -0.370762
DLi  cutpoint on noise_1 = -1.256350
DGi  cutpoint on noise_1 =  1.240120
```

So a differential assertion is meaningful. It is still asserted for **FIT vs DLi only**: `DLi` and `DGi` are two distances between the same pair of model-implied indicator VCVs, nothing stops them selecting the same candidate, and before P5 changed how node fits are estimated they did exactly that (both cut at `-1.69654`). Three-way distinctness would be an assertion about this fixture rather than about the kernels.

A note on injection. Now that B2 has made `splitter` a `match.arg()` string, a test can no longer pass a custom kernel by value. Anything that needs to observe or subvert a kernel must go through `local_mocked_bindings()`, which rebinds the symbol in the package namespace where `doTrees()`’s `switch()` resolves it. Verified working on `testthat` 3.3.2 (installed): the mocked counting wrapper below records the same 9 calls the by-value version did.

5.1 Kernel unit tests — the layer that was missing

Status: implemented (`tests/testthat/test-helper_doTrees.R`).

One MGA fit each, and the failure message names the broken kernel. This is the test that would have caught the missing `ndt_dists()` in seconds:

```

# As shipped (test-helper_doTrees.R). `args_trees` is the argument list of the
# csem() call the tree is built from -- see P5; it replaced the model string
# every one of these call sites used to take.
node_fixture <- function(max_cuts = 8L, minbucket = 100L) {
  ind <- parseModel(args_trees$.model)$indicators
  ft <- try_fit(dat[, ind, drop = FALSE], args_trees)
  list(model = list(object = ft$fit),      # what the trafo hands the kernel
        mf    = dat,
        subset = seq_len(nrow(dat)),
        ctrl  = list(collector = new_collector(), args = args_trees,
                      indicators = ind, max_cuts = max_cuts,
                      minbucket = minbucket))
}

test_that("every split kernel returns a finite statistic", {
  fx <- node_fixture()
  goes_left <- fx$mf$noise_1 <= stats::median(fx$mf$noise_1)
  for (k in c("FITdiff", "DLi", "DGi")) {
    expect_true(
      is.finite(partition_stat(k, fx$model, fx$mf, fx$subset, goes_left, fx$ctrl)),
      info = k
    )
  }
})

```

`partition_stat()` converts a missing-function error into NA + `n_fail_resample++`, so `is.finite()` catches it. Observed values at the median split are FITdiff = -0.00398, DLi = 0.0439, DGi = 0.0136.

The negative FIT difference got its own regression test. A two-group refit can fit **worse** than the pooled model, and on a noise covariate usually does — which is exactly the case `partition_select()`'s `.Machine$double.eps` floor exists to handle, since it reports statistics on the log scale and `log()` of a negative number is NaN. A NaN there does not error: it leaves that covariate's criteria column NA, so the covariate silently drops out of variable selection, and only ever for the covariates the data says least about.

```

test_that("partition_select() floors a negative argmax statistic", {
  fx <- node_fixture(); j <- match("noise_1", names(fx$mf))
  local_mocked_bindings(
    # Stubbed so the floor is what is under test rather than the fits: a real
    # scan takes the argmax, which is not guaranteed to be the negative case.
    scan_covariate = function(...) list(stat = -0.004, split = NULL,
                                         goes_left = rep(TRUE, length(fx$subset))),
    permutation_pvalue = function(...) 0.01
  )
  crit <- partition_select("FITdiff", split_max_fitdiff, fx$model, fx$mf,
                          fx$subset, j, fx$ctrl)$criteria
  expect_identical(crit["statistic", j], log(.Machine$double.eps))
  expect_true(is.finite(crit["p.value", j]))
})

```

Stubbing both helpers means this block runs no model fits at all. It is also the one test in the group that asserts an exact value rather than a property: $\log(.Machine$double.eps) \approx -36.0437$ is what the floor produces, and nothing else in the file would notice it becoming NaN.

5.2 `argmax_split()` returns an admissible cutpoint

Status: implemented (tests/testthat/test-helper_doTrees.R), as `one_test_that()` rather than the two this document originally proposed. The admissibility test already has all three cutpoints in hand when it finishes, so a separate “different kernels place different cutpoints” block would have re-run 16 two-group MGA fits to recompute two numbers it had just computed — four seconds for no coverage that the merged block does not already carry.

```
test_that("argmax_split() returns an admissible cutpoint the kernels disagree on", {
  fx <- node_fixture(); j <- match("noise_1", names(fx$mf)); z <- fx$mf[[j]]
  cutpoint <- function(kern) {
    sp <- argmax_split(kern, fx$ctrl$collector, fx$model, fx$mf,
                      fx$subset, j, fx$ctrl)
    # NULL is what argmax_split() returns when no candidate produced a finite
    # statistic -- the dead kernel this layer exists to localise.
    expect_true(inherits(sp, "partysplit"))
    partykit::breaks_split(sp)
  }
  brs <- vapply(list(FIT = split_max_fitdiff, DLi = split_max_dli,
                    DGi = split_max_dgi), cutpoint, numeric(1))
  for (k in names(brs)) {
    # Strictly inside the observed range: an endpoint sends every row one way.
    expect_true(is.finite(brs[[k]]) && brs[[k]] > min(z) && brs[[k]] < max(z),
              info = k)
  }
  expect_false(isTRUE(all.equal(brs[["FIT"]], brs[["DLi"]]))) # see fact (c)
})
```

This calls the kernels directly rather than through `doTrees()`, so it is unaffected by B2 and needs no mocking. It is also where cutpoint choice **must** be tested, per fact (b) — at the tree level the root gives the kernel no freedom, because `z_true` offers exactly one candidate.

Two details the naive version gets wrong. `inherits()` rather than `expect_s3_class()`, and the range check as a single `expect_true(..., info = k)` rather than two, because only the `info` form names the kernel that broke — which is the entire reason this layer exists rather than an assertion at the tree level. And the endpoint exclusion is strict on both sides: a cutpoint at `min(z)` or `max(z)` is one partykit accepts and grows a degenerate child from, so `is.finite()` alone would not catch it.

5.3 Assert that partykit actually calls the splitfun

Status: implemented (tests/testthat/test-postestimate_doTrees.R, next to the negative controls of 5.4 — both are about the kernel wiring rather than about any one configuration).

```
test_that("doTrees() installs its splitfun into partykit's split search", {
  calls <- new.env(parent = emptyenv()); calls$n <- 0L
  # Capturing the real kernel first is what stops the mock recursing into itself.
  real <- split_max_fitdiff
  local_mocked_bindings(
    split_max_fitdiff = function(model, mf, subset, goes_left, ctrl) {
      calls$n <- calls$n + 1L
      real(model, mf, subset, goes_left, ctrl)
    }
  )
  set.seed(11)
  tr <- grow_tree("mat", "FIT", ctl_mixed)
  expect_gt(calls$n, 0L) # the contract
  expect_grew(tr)        # ... and the tree is one the kernel actually shaped
})
```

Verified: `calls$n == 9, width == 3, n_split_scan == 2`, on partykit 1.2.29 and testthat 3.3.2. `expect_grew()` rather than the bare `width() > 1` first proposed: it also pins `n_fail_split == 0`, so a mock that fired but returned nothing usable — 9 calls and a stump — cannot pass. This test is what breaks loudly if a partykit upgrade stops honouring `ctrl$splitfun`, which would otherwise show up only as a silent fallback to partykit’s own `maxstat` scan.

5.4 Negative control — give the suite teeth

Status: implemented and passing (tests/testthat/test-postestimate_doTrees.R, 10 assertions across three blocks). One test alone would not have been enough: asserting only that a dead kernel **warns** leaves open whether a working kernel warns too, so the trio brackets the behaviour from both sides plus the kernel-free path.

As shipped, after T2–T4 hoisted `ctl_mixed`, `grow_tree()` and `expect_grew()` to the top of the file:

```
test_that("a dead split kernel is reported, not silently a stump", {
  local_mocked_bindings(
    split_max_dli = function(model, mf, subset, goes_left, ctrl) {
      stop("kernel is broken")
    }
  )
  set.seed(11)
  expect_warning(
    tr <- grow_tree("mat", "DLi", ctl_mixed),
    "produced no usable statistic"
  )
  info <- attr(tr, "igsca_info")
  expect_gt(info$n_fail_split, 0L)
  expect_identical(info$n_fail_split, info$n_split_scan)
  # The tree really is the stump the diagnostic is warning about
  expect_equal(partykit::width(partykit::node_party(tr)), 1)
})

test_that("a working split kernel records scans but no failures", {
  set.seed(11)
  tr <- expect_no_warning(grow_tree("mat", "DLi", ctl_mixed))
  expect_gt(attr(tr, "igsca_info")$n_split_scan, 0L)
  expect_grew(tr)
})

test_that("the native split path never touches the kernel counters", {
  set.seed(11)
  tr <- grow_tree("mat", "native", ctl_mixed)
  info <- attr(tr, "igsca_info")
  expect_identical(info$n_split_scan, 0L)
  expect_identical(info$n_fail_split, 0L)
})
```

The middle block is the positive control for `expect_grew()` itself: it runs the same configuration as the first, unmocked, so between them the pair pins that the warning and the counters track the kernel rather than the control. Two details worth carrying: the reduced `ctl_mixed` keeps all three blocks to a few seconds, which costs nothing in fidelity since mixed splitters run no permutation test (T4); and `partykit::width()` returns a **double**, not an integer, so `expect_identical(width(...), 1L)` fails on type alone — use `expect_equal(..., 1)` or `expect_gt()`.

5.5 The scan cache

Status: implemented (tests/testthat/test-helper_doTrees.R, with a `node_fixture()` that packages one pooled node fit the way the trafo hands it to a kernel).

`argmax_split()`'s cache key has two components — `identical()` on the splitter **closure** and `identical()` on the subset (R/helper_doTrees.R:294--296) — so the test exercises both. A matched selector/splitter pair reuses the selector's scan; a mismatched kernel, or the same kernel at a different node, rescans:

```
sc <- scan_covariate("FITdiff", j, fx$model, fx$mf, fx$subset, fx$ctrl)
coll$scan <- stats::setNames(list(sc), as.character(j))
coll$scan_subset <- fx$subset
coll$scan_splitter <- split_max_fitdiff

expect_identical(                                     # matched -> cache hit
  argmax_split(split_max_fitdiff, coll, fx$model, fx$mf, fx$subset, j, fx$ctrl),
  sc$split)
expect_identical(coll$n_split_scan, 0L)                # ... kernel never ran

sp_dli <- argmax_split(                                # mismatched -> rescan
  split_max_dli, coll, fx$model, fx$mf, fx$subset, j, fx$ctrl)
expect_identical(coll$n_split_scan, 1L)
expect_false(isTRUE(all.equal(                        # and it disagrees
  partykit::breaks_split(sp_dli), partykit::breaks_split(sc$split))))

coll$n_split_scan <- 0L                               # same kernel, other node
argmax_split(split_max_fitdiff, coll, fx$model, fx$mf, fx$subset[-1L], j, fx$ctrl)
expect_identical(coll$n_split_scan, 1L)
```

The `n_split_scan` assertions are the load-bearing ones. Comparing the returned splits alone would not distinguish a hit from a rescan that happened to land on the same cutpoint — and on the matched pair it **always** would, by construction. An unchanged counter is what “the kernel never ran” actually means.

The differing-cutpoint assertion is a bonus that holds because of fact (c) above: FIT cuts `noise_1` at `-0.370762`, DLI at `-1.256350`. Do not extend it to `DLi × DGi`, which may legitimately agree — 5.2 makes the same FIT-vs-DLi comparison and stops at the same place, for the same reason.

Note this test must **not** be combined with `local_mocked_bindings()` on the same kernels: the cache key is closure identity, and a mock would change it. That is also why the `n_fail_resample` test next to it mocks `try_fit` rather than a kernel.

5.6 What none of this proves

The five layers above establish that the mixed splitters **run, are invoked, and discriminate**. They do not establish that a mixed splitter finds the **right** cutpoint, because this fixture has no numeric covariate with a true threshold — `z_true` is binary and `noise_*` are noise. A recovery test needs a fixture where a numeric covariate drives a known group difference at a known cutpoint, then asserts the recovered breakpoint falls within a tolerance of it. That is the only test that checks the mixed configurations are statistically correct rather than merely alive, and it is worth building if the mixed pairs are a real study arm rather than a convenience.

6 Packaging and API

6.1 P1. The export list is now four symbols and two methods

Status: fixed. The first pass complained that `NAMESPACE` was stale; running `document()` inverted the problem into 26 exported symbols, most of them internal machinery with no standalone meaning, each one a public commitment and each one carrying a `man/*.Rd` for R CMD check to scrutinise (P3).

22 of them are now `@noRd`, which deletes the export **and** the `.Rd` file. What remains is the surface this section proposed:

Exported	Why it is public
<code>doTrees()</code>	The entry point
<code>igsca_tree_control()</code>	Its <code>.control</code> argument; useless if unreachable
<code>bdiagFit()</code>	A general utility, already documented, already referenced from <code>zz_arguments.R</code> , and already tested
<code>root_criteria()</code>	Reads a diagnostic off a returned tree
<code>coef()</code> / <code>plot()</code> methods	S3 dispatch on <code>igsca_tree</code>

Un-exporting cost the tests nothing, which is the point worth recording: a testthat file runs with the package namespace as its parent, so `test-helper_doTrees.R` still calls `partition_stat()`, `scan_covariate()`, `argmax_split()` and the `split_max_*` kernels directly, and `local_mocked_bindings()` still rebinds them — it operates on the namespace, not the export list. PASS 150 | FAIL 0 after the pruning, unchanged.

6.2 P2. `:::` and `::` into the package’s own namespace

Status: fixed. Three call sites dropped their prefixes:

```
Sc = bdiagFit(single_fit, .n_blocks = 2L, .type_vcv = "construct"),
Si = bdiagFit(single_fit, .n_blocks = 2L, .type_vcv = "indicator")
...
calculateFIT(mga$fit) - calculateFIT(model$object)
```

Both functions are defined in this package (`bdiagFit` in `helper_doTrees.R`, `calculateFIT` in `helper_assess.R`). R CMD check flags the `:::` form (“a package almost never needs to use `:::` for its own objects”), and both forms break under `load_all()` when the installed and sourced versions diverge — a hazard this port hit for real, since `calculateFIT()` is exactly where P5’s silent PLS-PM refit surfaced.

The inconsistency was internal to one file: `ndt_dists()` already called `bdiagFit()`, `calculateDG()` and `calculateDL()` bare and correctly, 390 lines below `ndt_pools()` doing the same thing through `cSEM:::`.

Two stale comments went with it: the `COIN_ssr` naming that `influence_ssr` → `influence_vec` left behind, and the registry comment still describing the methods as `igsca_ctree()` entry points.

6.3 P3. Roxygen placeholders — resolved by pruning, then by writing them

Status: fixed. `document()` had turned the placeholder blocks into committed documentation. `man/idx_permutation.Rd` in full, as it stood:

```
\title{Title}
\usage{idx_permutation(n, R, strata = NULL)}
```

```

\arguments{
\item{strata}{}
}
\description{
Permutation resampling: sample WITHOUT replacement, within each stratum, ...
}

```

Three R CMD check problems in nine lines: the title is literally `Title`, `n` and `R` are undocumented while `strata` has an empty description, and there is no `\value{}`. That pattern repeated across the 20-odd new `.Rd` files.

It was fixed in the order this section recommended. P1's pruning deleted **23** `.Rd` files outright, taking their warnings with them; 43 bare `@param` placeholders went with them, since roxygen warns about an empty `@param` even inside an `@noRd` block. The internal blocks kept their prose and lost only their tags — a `#' Title` line above a real description was simply removed, promoting the description to the title position, and the dozen blocks that had **no** description now have a one-line one.

The four survivors were then written properly: `igsca_tree_control()` documents all eight arguments, including the two whose failure modes this document had to discover (`minbucket` admitting unfittable leaves, `R_test` below 20L making splitting impossible); `root_criteria()`, `coef.igsca_tree()` and `plot.igsca_tree()` document their arguments and returns.

`devtools::check_man()` is now clean for every file this port touches. The remaining warnings are pre-existing and unrelated — unresolved links in `00_csem.R` to `parseModel`, `cSEMModel` and the `calculateWeights*` internals.

Note the `strata` argument in the `.Rd` above is also gone: B1 flagged it as an untested branch `permutation_pvalue()` never used, and `idx_permutation(n, R)` now takes only what it is called with.

Two specific errors in `man/doTrees.Rd`, **both fixed** alongside P5's signature change:

- **\value was wrong.** It claimed class `modelparty` and `party`. The actual class is `c("igsca_tree", "constparty", "party")` on both paths; there is no `modelparty` anywhere in the package.
- **Half the arguments were undocumented.** `data`, `model` and `covariates` did inherit from `csem_arguments` (roxygen matched them to the dot-prefixed `.data` / `.model` / `.covariates` entries). `influence`, `splitter` and `control` did not — they appeared nowhere in `\arguments{}`, which is an R CMD check WARNING. They are also the three arguments a reader most needs explained, since `.influence` selects between two entirely different algorithms and `.splitter` is constrained by which one you picked (B2's `stopifnot`). All five are now documented explicitly rather than inherited.

Both were fixed by writing the five arguments out explicitly rather than inheriting them.

6.4 P4. Argument naming departs from the package convention

Status: fixed, together with P5 — the signature was breaking anyway, so both landed in one change. `doTrees(.object, .covariates, .influence, .splitter, .control)` now matches every other exported cSEM entry point.

(The first pass claimed the bare names also broke `@inheritParams`; that was wrong — roxygen matched the dotted entries to the undotted parameters, as P3 shows. The case for renaming rested on API consistency alone.)

6.5 P5. The estimator is no longer chosen by `fit_csem()` at all

Status: fixed. `doTrees()` used to take `data + model` and refit every node through a `fit_csem()` that hard-coded GSCA, CCMP, `.iter_max = 100` and `.tolerance = 1e-4`, with no `...` and no argument to get past it.

It now takes a **fitted** object and replays that fit's own arguments:

```
doTrees(.object, .covariates, .influence, .splitter, .control)

fit_csem <- function(.data, .args, .id = NULL) {
  if (!is.list(.args)) stop2(...)
  .args$.data <- .data
  .args$.id <- .id
  do.call(csem, .args)
}
```

`.args` is `.object$Information$Arguments`, so the data, the model, the estimator, the modes and the convergence settings all arrive from one place and cannot drift apart. Three measurements made this design possible:

- **`csem()` stores the whole input `data.frame`** — non-indicator columns and factors included, and it ignores them when fitting. So the covariates ride along in the original call, `doTrees()` needs no `data` argument, and the rows it partitions are by construction the rows the model was fitted on.
- **The stored `.model` is a parsed `csemModel`**, and `parseModel()` round-trips it identically, so it can be handed straight back to `csem()`. Re-passing the model turned out to be unnecessary.
- **`do.call(csem, args)` reproduces the original fit exactly**, and a node refit built this way is identical to what the hard-coded call produced — verified on node sizes 400, 500 and 137 (paths equal), and on the two-group MGA fit. The suite's snapshots did not move.

Two traps worth recording, both silent.

`modifyList()` is the obvious way to swap `.data`, and it is wrong. It recurses whenever old and new values are both lists — and a `data.frame` is a list — so it merges the node's columns into the stored 1000-row data instead of replacing it. Worse, `[[<- .data.frame` **recycles** rather than errors when the node size divides the original, so a 500-row node fits happily on a duplicated copy of itself and only a node of, say, 137 rows reveals the bug.

`$<-` coerces an atomic to a list. A call site still passing the model **string** therefore produced a list whose unnamed first element matched `csem()`'s `.model` by position — yielding a fit that ran, converged, and silently used **PLS-PM**, `csem`'s default. It surfaced only as `calculateFIT()` erroring into an NA deep inside `partition_stat()`, i.e. as a stump. `fit_csem()` now rejects a non-list `.args`, and a test asserts the leaf fits' `.approach_weights` on **both** branches — the ctree-only version of that test missed this exactly.

What this buys. The tree machinery is estimator-agnostic wherever the statistics are: `DLi/DGi` read model-implied indicator VCVs and now work on a PLS-PM fit. The residual (`mat/vec`) and FIT statistics are not — `calculateGSCAErrors()` returns NA rather than erroring off GSCA, and `calculateFIT()` errors — so those combinations are refused up front rather than left to fail deep inside. See P6.

6.6 P6. No input validation

Status: fixed for the new contract. `doTrees()` used to validate `influence` and `splitter` (via `match.arg`) and nothing else. P5 removed two of the original three cases outright — the data and

the indicators now come from the fitted object, so they cannot disagree with each other — and the rest are checked in two `@noRd` helpers:

- `csem_tree_args()`: `.object` must be a `cSEMResults`, and must not be `cSEMResults_multi`. The multigroup refusal is the substantive one: such an object keeps its `Information` per group and has no top-level `Arguments` to replay, and grouping is what the tree is **for**.
- `validate_tree_input()`: `.covariates` must be non-empty, must be columns of the object’s data (the message says to include them in the `csem()` call, since non-indicator columns are ignored when fitting), and must not also be indicators of the model. Plus the estimator check of P5.

(`stop2()` is the package’s own error helper, `R/zz_utils.R:114`.)

The five blocks in `test-postestimate_doTrees.R`’s “Input contract” section cover all of it, including the one case that is **allowed**: a $DLi \times DLi$ tree on a PLS-PM fit.

7 What the port got right

Worth recording, since these are deliberate improvements over the template rather than accidents:

- **Indicators from the model, not the data.** `parseModel(model)$indicators` (R/postestimate_doTrees.R:33) replaces the template’s `setdiff(names(data), covariates)`. The template silently treated any stray column as an indicator; the port does not, and now tolerates extra columns in `data`. The two orderings differ (`x31, x32, ..., x11` vs `x11, x12, ...`), which is why the recorded snapshot’s model formula reads `~x31 + x32 + x33 + x11 + ...` — expected, not a bug.
- **The unified dispatch is the right cut.** Branching on `influence` rather than exposing two constructors correctly reflects that the COIN and partition families differ only in how the node statistic is produced.
- **`idx_permutation()` and `ndt_dists()` were ported carefully.** The $R \times n$ orientation and the (`Sc_pool`, `Si_pool`, `mga_fit`) argument order both match their call sites, and `ndt_dists()` uses unprefix internal calls — the convention the older `ndt_pools()` still violates.
- **B2 was fixed properly rather than patched.** Making `splitter` a `match.arg()` string closed the `logical(0)` hole structurally instead of special-casing `NULL`, and kept the one `stopifnot()` that still carries meaning. Both guard paths now produce errors a user can act on.
- **`influence_ssr` → `influence_vec`** reads better next to `influence_mat`. The stale “COIN_ssr” references in the comments (R/helper_doTrees.R:113, :514) should follow the rename, as should the now-incorrect provenance comment at :535.
- The scan cache, minbucket filtering, and `.Machine$double.eps` flooring all survived the port intact and behave correctly under test.

8 Suggested order of work

#	Why in this position
1	Done. ~ 16 h → 2 min 05 s — two controls; chosen by branch
2	Done. 85 assertions, FAIL 0 WARN 0 T3 — regenerate snapshots; drop length()==5; split expect_no_error()
3	Done. PASS 167 across the two files; 5.2’s two proposed blocks merged into one, and 5.1 gained the eps-floor regression test it flagged — kernel; argmax_split; and splitfun-

#	Why in this position
	wiring tests
4	Done. Two-part cache key both covered; PASS 121 across the two files + \$5.5 — collector and scan- cache tests
5	Done. 26 exports → 4 + 2 methods, 23 .Rd files deleted, check_man() clean for every file He port touches — prune exports; drop ::+; fix \value; validate inputs
6	fail_leaf is now asserted (T5), but the default 30L still admits leaves the model cannot follow the assertion pins the symptom, it does not fix it up — raise minbucket for large models
7	Optional ; only if the mixed pairs are a real study arm — numeric- cutpoint fixture
8	Done. The override was measured to be a no-op on all four combinations and deleted with both TODOs replace the cc\$bonferroni override with a length-2 testtype

9 Design record: the fitted-object API

Written after the fact, at the point of implementation (commit 3131b5d5), so that the reasoning survives if the decisions have to be revisited. This records **why** the alternatives were rejected, not what was built — for that see P5.

9.1 The problem

`doTrees(data, model, covariates, ...)` refit every node through a `fit_csem()` that hard-coded `.approach_weights = "GSCA"`, `.disattenuate = TRUE`, `.GSCA_modes = "CCMP"`, `.conv_criterion = "sum_diff_absolute"`, `.iter_max = 100` and `.tolerance = 1e-4`. There was no `...` and no argument to reach past it, so the tree's estimator was a property of the **package**, not of the analysis. Threading `...` through would have fixed the immediate complaint while leaving the user to keep two descriptions of the same model in sync — the one they fitted with, and the one the tree refits with.

Taking a **fitted object** removes the second description entirely.

9.2 The three decisions

(1) **Where do the covariates come from?** Chosen: the fitted object's own stored data; `doTrees()` has no `data` argument.

Option	Why it was or was not taken
From .object only → chosen	<code>csem()</code> stores the whole input <code>data.frame</code> and ignores non-indicator columns when fitting (measured), so covariates ride along at no cost. The decisive property is not convenience but row alignment : the rows the tree partitions are by construction the rows the model was fitted on, and no argument exists through which they could disagree
Separate <code>data</code> argument	Would suit a user who fitted <code>csem()</code> on indicator columns only. Rejected: nothing checks that <code>data</code> 's rows correspond to the fit's, and a silent row mismatch is precisely the class of failure this port keeps producing (see S1, and <code>modifyList()</code> in P5)
<code>.object</code> by default, <code>data</code> as override	Both behaviours, two code paths to document and test, and the override reintroduces the mismatch risk for the one case it serves. YAGNI

(2) **Argument naming.** Chosen: dot-prefix everything

— `.object`, `.covariates`, `.influence`, `.splitter`, `.control`. The signature was breaking regardless, so this was the cheapest possible moment to close P4 deferring it would have meant breaking the signature twice.

(3) **A .model override?** Chosen: no argument at all; the fit's model is always reused.

This was the decision most likely to go the other way, and the measurement settled it. The stored `.model` is a parsed `cSEMModel` and `parseModel()` round-trips it identically, so it can be handed straight back to `csem()` — the anticipated need to **re-pass** the model turned out not to exist. An override would only serve fitting a **different** model in the nodes than at the root, which makes the returned object describe two models at once. If that is ever wanted, it should be a different function, not an argument.

9.3 What would force a revisit

- **A user who cannot refit.** The whole design assumes `csem()` can be re-run on subsets. It is already true for every node the tree grows, so this is not a new constraint — but a fit whose arguments cannot be replayed (a hand-modified `cSEMResults`, or a future `csem()` argument that is not idempotent) would break it silently. `fit_csem()`'s `is.list()` guard catches only the crudest version of that.
- **Second-order or multigroup input.** Both are refused today. Multigroup is refused on principle (grouping is the tree's job); second-order is refused only incidentally, by `calculateFIT()`'s own `stop2()`. If 2nd-order trees are ever wanted, the refusal needs to become explicit and the FIT statistics need a story.
- **A non-GSCA study arm.** The estimator guard is deliberately narrow: it refuses only the combinations whose statistics are GSCA-specific (`mat/vec/FIT`), and lets `DLi/DGi` through on any estimator. If a PLS-PM tree ever becomes a real study arm, §5.6's recovery fixture is what would establish that those distances behave, since nothing currently checks they are **statistically** meaningful off GSCA — only that they run.

9.4 What the process caught

Both traps in P5 — `modifyList()`'s recursive merge and `$<-`'s coercion of a model string into a positional `.model` — produced fits that **ran and converged**. Neither was found by reading; the first by testing a node size that does not divide the original (137), the second by the suite's snapshots refusing to reproduce. Both were introduced by this change and caught before it landed, which is the argument for having done T1–T5 first.